



```
#ifndef __PROGBASE_2019__  
#define __PROGBASE_2019__
```

CPU/ProgBASE 2019 – Informações Gerais

A) Sobre a entrada:

- 1) A entrada de seu programa deve ser lida da entrada padrão.
- 2) A entrada é composta por vários casos de teste, cada um descrito em um número de linhas que depende do problema.
- 3) Quando uma linha da entrada contém vários valores, estes são separados por um único espaço em branco; a entrada não contém nenhum outro espaço em branco.
- 4) Cada linha, incluindo a última, contém o caractere final-de-linha.
- 5) O final da entrada coincide com o final do arquivo.

B) Sobre o seu código:

- 1) O nome do arquivo de seu programa é o nome do problema seguido da extensão da linguagem escolhida.
- 2) O seu programa deve ser escrito em uma das linguagens permitidas: Java, C, C++ ou Python 2 ou 3.
- 3) Um problema pode ser resolvido em uma linguagem diferente de outro, se desejar.
- 4) O seu programa deve ler a entrada do console (entrada padrão).
- 5) O seu programa deve gerar a saída em console (saída padrão).

C) Sobre a saída:

- 1) Quando uma linha da saída contém vários valores, estes devem ser separados por um único espaço em branco; a saída não deve conter nenhum outro espaço em branco.
- 2) Cada linha, incluindo a última, deve conter o caractere final-de-linha.

D) Sobre a competição:

- 1) Linguagens aceitas:
 - a) C, extensão “.c” → `gcc -O2 -lm`
 - b) C++, extensão “.cpp” → `g++ -std=c++11 -O2 -lm`
 - c) Java, extensão “.java” → `(LC_ALL=en_US.UTF-8 java ..)`
 - d) Python 2, extensão “.py2”
 - e) Python 3, extensão “.py3”
- 2) As questões, embora escritas de forma original pela equipe organizadora, podem ser baseadas em problemas de mesma categoria já existentes. Em específico este caderno, em parte, está baseado em questões passadas da Maratona de programação SBC/ACM e escritos de Olavo B. Amaral, PhD.

Problema A
Ar-condicionado Ideal
Arquivo: ar.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

Você é um instalador de ar-condicionado, e sempre lhe perguntam qual é o ideal para uma determinada sala. A resposta depende do tamanho da sala, pois dada sua área o ar-condicionado pode ser:

Área (m^2)	Capacidade (BTUs/h)
Até 12	9.000
Até 16	12.000
Até 24	18.000
Até 29	22.000
Até 32	24.000
Até 37	28.000
Até 40	30.000
Acima de 40	Ar-condicionado Industrial

Para facilitar a sua vida, o instalador lhe pediu um programa que, dadas as dimensões da sala, já indique a capacidade ideal do ar-condicionado.

Entrada:

Cada caso de entrada representa um caso de teste. A entrada é representada por dois valores (**C** e **L**) de ponto flutuante representando o comprimento e largura da sala ($1.5 \leq C, L \leq 10$) em metros.

Saída:

Para cada caso de teste deve-se imprimir em uma linha única a capacidade do ar-condicionado ideal, com o ponto de separação de milhar, seguido de um espaço e da sigla “BTUs”. Ou a informação “Ar-condicionado Industrial”, caso não exista solução prática para o tamanho da sala. Sempre seguido de final de linha.

Exemplo de entrada:

Saída do Exemplo de entrada:

4 4	12.000 BTUs
3.5 5.5	18.000 BTUs
6 7	Ar-condicionado Industrial

Problema B
Bora, Quadrilha Animada!
Arquivo: quadrilha.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

Durante o período das festas juninas, no interior de São Paulo, as escolas de ensino fundamental sempre preparam uma festa com as crianças, incluindo a dança da quadrilha. Para a dança da quadrilha, as crianças são organizadas em pares, meninos e meninas, para dançar as músicas. O problema é que nem todos os meninos e meninas topam formar qualquer par, pois já na tenra idade existem afinidades e desafetos. Conhecendo a turma, as “tias” já fazem um esquema representando quem aceita dançar com quem. E precisam de um programa que diga se é possível formar um conjunto de pares com todos os meninos e meninas, ou não (sempre tem o mesmo número de meninos e meninas).

Entrada:

Cada caso de teste apresenta várias linhas, a primeira linha contém dois inteiros, sendo **N** o número de meninos que é igual ao número de meninas ($0 < \mathbf{N} \leq 20$), e **M**, as ligações de afinidades ($0 \leq \mathbf{M} \leq \mathbf{N}^2$). As próximas **M** linhas contêm as ligações de afinidade indicadas por dois inteiros: **ELE** e **ELA** ($1 \leq \mathbf{ELE}, \mathbf{ELA} \leq \mathbf{N}$). Indicando que o *i*-ésimo menino tem afinidade com a *k*-ésima menina (todos os meninos e meninas receberam um número de 1 a **N**). Os casos de teste se encerram com **N** = **M** = 0.

Saída:

Para cada caso de teste deve ser impressa em uma única linha a frase: “Deu quadrilha”, se foi possível formar os pares com todos os meninos e meninas, ou “Deu ruim”, se não foi possível, sem aspas.

Exemplo de entrada:

```
6 10
1 1
2 3
2 5
3 1
3 2
4 3
5 1
5 6
6 4
6 6
6 10
1 1
2 3
3 1
3 4
3 6
4 3
5 2
5 4
5 5
6 6
0 0
```

Saída do exemplo de entrada:

```
Deu quadrilha
Deu ruim
```

Problema C
Consigo Férias Econômicas?
Arquivo: ferias.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

Seu colega adora passar as férias em hotéis que oferecem diversos recursos como entretenimento. Só que estes hotéis, além da diária, cobram a cada dia o custo do entretenimento do dia, que pode ser até gratuito, porém, considerando a diária, cada dia tem um custo não nulo. Sabendo de antemão o custo de cada dia, em sequência, do período de férias, e o máximo de dinheiro que dispõe para sua diversão, seu colega quer saber o período máximo de dias consecutivos que pode se hospedar no hotel gastando no máximo o dinheiro que reservou para estas férias.

Por exemplo, se ele tiver 8 dias disponíveis, e o custo planejado para cada dia for: $\{30, 58, 43, 65, 33, 42, 50, 35\}$, e somente dispuser de R\$160,00, ele poderá viajar do quinto ao oitavo dia, ou seja 4 dias no máximo em suas férias, dentro deste orçamento.

Claro que se não viajar – 0 dia –, vai gastar 0 real. Para facilitar sua vida, ele pediu a você que fizesse um programa para identificar quantos dias ele poderá ficar no máximo.

Entrada:

A entrada possui vários casos de teste. Cada caso de teste é composto por 2 linhas. A primeira linha contém dois valores inteiros: N ($0 < N \leq 300$), a quantidade de dias em sequência que poderá tirar férias, e R ($0 \leq R \leq 10^4$), o valor disponível para a viagem. Na segunda linha são N números inteiros, indicando o custo em sequência para cada dia i ($1 \leq i \leq N$) das férias, sendo que cada um é um valor positivo não maior que 100. Os casos de teste terminam com $N = R = 0$, que não deverão ser considerados.

Saída:

Para cada caso de teste espera-se em uma linha única o número máximo de dias em sequência que seu colega poderá viajar naquele período.

Exemplo de entrada:

```
8 160
30 58 43 65 33 42 50 35 5 50
29 37 43 18 33
1 80
100
0 0
```

Saída do exemplo de entrada:

```
4
1
0
```

Problema D

Divisando Prédios no Horizonte

Arquivo: predios.[c|cpp|java|py2|py3]

Limite de tempo: 1 s

O Problema:

Você e seu colega estavam à janela do apartamento observando os prédios no horizonte, bem à frente vocês viram três prédios. Quando foram consultar o mapa predial do município, perceberam que à frente havia de fato 5 prédios, e o detalhe é que, na sequência, as alturas dos prédios eram de {20m, 40m, 10m, 30m, 50m}. Com esta ideia, seu colega o desafiou a implementar um algoritmo que indique quantos prédios podem ser vistos, a partir de uma lista das alturas dos prédios em sequência.

Entrada:

São vários casos de teste. Cada caso de teste usa duas linhas, na primeira linha, um inteiro $\mathbf{N}$ ($0 < \mathbf{N} \leq 10^7$) é a quantidade de prédios que está à frente. Na segunda linha são $\mathbf{N}$ valores inteiros positivos não maiores que 10^9 que são as alturas dos prédios. Os casos de teste terminam quando $\mathbf{N}=0$, que não deve ser tratado.

Saída:

Para cada caso de teste, deve ser impresso, em uma linha única, quantos prédios estão à vista no horizonte para a sequência.

Exemplo de entrada:

```
5
20 40 10 30 50
5
10 20 30 40 50
5
50 40 30 20 10
0
```

Saída do exemplo de entrada:

```
3
5
1
```

Problema E

Escolha o Lanche Personalizado

Arquivo: `lanche.[c|cpp|java|py2|py3]`
Limite de tempo: 1 s

O Problema:

Em seu bairro abriu uma lanchonete que serve lanche de metro personalizado. A lanchonete oferece **N** opções de recheio para o lanche, e permite que você escolha **P** destas opções, sendo que você pode repetir **P** vezes um recheio. Os valores de **N** e **P** dependem do dia e do movimento. No dia que você foi eram 6 opções e 3 recheios. Você ficou curioso sobre quantas formas diferentes você pode construir um lanche, e resolveu fazer as contas. Para aquele dia eram 56 possibilidades distintas. Para facilitar a contas nos outros dias, você resolveu fazer um algoritmo que calcule o número de lanches distintos que podem ser feitos a partir da quantidade de recheios e opções de escolha disponíveis.

Entrada:

São vários casos de teste. Cada caso de teste é representado por dois números inteiros **N** e **P**, com $0 < \mathbf{N} \leq 30$, e $0 < \mathbf{P} \leq 30$. Os casos de teste terminam com o fim das entradas.

Saída:

Para cada caso de teste, deve ser impresso, em uma linha única, quantos lanches distintos podem ser feitos a partir da combinação de entrada.

Exemplo de entrada:

```
6 3
8 5
5 2
10 1
```

Saída do exemplo de entrada:

```
56
792
15
10
```

Problema F
Forward/ backward/ to the left/ to the right
Arquivo: fblr.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

No Sudoeste amazônico houve um povo de localização especialmente fixa, diferente de outros de mesma linhagem que eram nômades. Avanços de arqueologia e análise de DNA mostram que os Ayahk viveram em um vale fértil, aproximadamente de 600 d.C. até 1821, quando os últimos morreram por varíola, por contato com exploradores hispânicos. Antes da extinção desse povo, um antropólogo francês de nome Henri-Marie Djobeaube viveu entre eles, os estudou, e registrou aquela que provavelmente foi a principal razão de por que os Ayahk não se tornaram em nômades: no seu idioma não existia o verbo “ir”, só o verbo “voltar”. Eles nunca diziam “Eu estou indo ao rio pescar”, e sim “Eu estou voltando para casa passando pelo rio onde antes pescarei”, mesmo ainda a um passo de sua porta. Os Ayahk instintivamente conceberam que uma linha reta não passa de uma abstração matemática, e que, alguém saindo de um ponto A, e caminhando em qualquer sequência de sentidos, necessariamente percorrerá uma curva ou sequência de curvas, e, assim, desde que haja tempo suficiente, em algum momento voltará ao ponto A, devido à curva ou à sequência de curvas, por menos aberta que seja. Claro, para certas sequências o tempo necessário tenderia ao infinito. Considere os sentidos cardeais **L** (Leste), **O** (Oeste), **N** (Norte) e **S** (Sul), e que cada percurso de volta de um ayahk é sempre uma sequência de curvas, cada uma dessas curvas da sequência sendo representada pela inicial do seu sentido. Cada curva tem o mesmo comprimento, e cada sequência tem tamanho mínimo de 2 curvas. Por exemplo: **NN**, ou **NS**, ou **SOLN**, ou **LOO**, ou **NLNLOSOS**, ou **SONL** etc. Ao analisar um percurso, queremos saber se houve a volta completa ou a mesma se interrompeu por qualquer motivo (uma onça almoçando? uma pausa longa demais?). O algoritmo deve verificar se determinada sequência de curvas fez o ayahk voltar ou não.

Entrada:

São vários casos de teste. Cada linha representa um caso de teste, ou seja, uma sequência de curvas formada por uma permutação das letras **L**, **O**, **N** e **S**, sem espaço. Os casos de teste terminam com o fim das entradas. Cada sequência tem no máximo 10^5 letra representando uma curva.

Saída:

Para cada caso de teste da entrada seu programa deve gerar uma única linha na saída, contendo a palavra “Voltou”, se sim, ou a frase “Deu zebra” em caso contrário.

Exemplo de entrada:

```
SOLN
NLNLOSOS
NN
SSLNNOOL
```

Saída do exemplo de entrada:

```
Voltou
Voltou
Deu zebra
Voltou
```


Problema G
Gabriela, mãe extremosa
Arquivo: gabriela.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

Gabriela era uma menina muito criativa e vivia inventando brincadeiras para se divertir com seus amigos. Ela cresceu, se casou e teve dois filhos, e está cada dia mais preocupada, pois eles não gostam muito de matemática, e só querem passar o dia jogando diversos jogos eletrônicos que têm. A dona Gabriela teve, então, uma ideia para associar o aprendizado aos jogos de que eles tanto gostam. Inventou um jogo chamado de Jogo da Soma. Porém, a dona Gabriela é advogada e não entende nada de programação de computador, e precisa de ajuda para transformar sua ideia em um jogo eletrônico. Neste problema o seu algoritmo vai “arbitrar” o jogo. Trata-se de um jogo de tabuleiro que é disputado entre dois jogadores: Jogador 1 e Jogador 2. O jogo pode ter um de três níveis: Fácil, Médio e Difícil, que deve ser escolhido antes de tudo mais. No nível Fácil o tabuleiro é de dimensão 2x2, no nível Médio de 3x3, e no Difícil de 4x4. O tabuleiro tem no começo todas as casas zeradas. O Jogador 1 joga com as linhas do tabuleiro, e o Jogador 2, com as colunas. O Jogador 1 inicia a partida. A cada rodada, o jogador da vez posiciona um número em uma das casas do tabuleiro. São permitidos somente inteiros do intervalo fechado entre 1 e 100. Cada número só pode ser usado uma única vez por partida. Alternam-se as jogadas entre os jogadores até que não haja mais casas zeradas. Calculam-se as somas de cada uma das linhas e de cada uma das colunas do tabuleiro, e, se a maior soma for de uma das linhas, o Jogador 1 é declarado vencedor; caso a maior soma seja de uma das colunas, é vencedor o jogador 2. Em caso de empate, verifica-se a segunda maior soma e assim por diante. Se o empate persistir para todas as possibilidades, este é declarado.

Entrada:

São vários casos de teste. Cada caso de teste contém: 1) Uma linha com apenas a letra que representa o nível dessa partida (**F**, **M** ou **D**); 2) Todas as jogadas (uma a cada linha a seguir) em quantidade determinada pelo nível da partida. Uma jogada é composta por 3 inteiros separados por um espaço em branco. O primeiro e o segundo caracterizam a posição do tabuleiro escolhida pelo jogador da vez (o primeiro inteiro, a linha e o segundo inteiro, a coluna), e o terceiro inteiro é o valor jogado naquela posição. As posições do tabuleiro variam de [1, 1] a [N, N], **matricialmente**, com N a depender do nível da partida. A entrada termina com o final de arquivo.

Saída:

Para cada caso de teste, seu programa deve imprimir uma única linha, com os termos **X**, **Y** e **Z** separados por um espaço em branco. Sendo **X** igual a 1 ou 2, a depender do jogador vencedor; **Y**, o maior total de linha dentre todos os totais de linha e **Z**, o maior total de coluna dentre todos os totais de coluna, em caso de vitória do Jogador 1, ou vice-versa; ou ainda a frase “Houve empate em todas as possibilidades”.

Exemplo de entrada:

```
F
1 1 3
2 2 4
1 2 2
2 1 1
M
2 2 7
1 3 9
3 1 8
1 2 5
2 3 2
3 2 3
2 1 6
1 1 1
3 3 4
D
1 1 100
2 2 99
3 3 98
4 4 97
1 2 96
2 1 95
3 4 94
4 3 93
1 3 92
```

2 4 91
3 1 90
4 2 89
1 4 88
2 3 87
3 2 86
4 1 85

Saída do exemplo de entrada:

2 6 5
Houve empate em todas as possibilidades
1 376 370

Problema H

Há vagas!

Arquivo: *vagas*.*[c|cpp|java|py2|py3]*

Limite de tempo: 1 s

O Problema:

Neste problema, o algoritmo a ser implementado simula um autoestacionamento. Os carros entram pela extremidade Sul do autoestacionamento e saem pela extremidade Norte, necessariamente. Se chega um cliente para retirar um carro que não esteja estacionado na posição extremo-norte, todos os carros à frente desse carro saem do estacionamento e entram de novo, em ordem. Sempre que um carro sair definitivamente, todos os carros são deslocados para a frente, de modo que todos os espaços vazios estarão sempre na extremidade Sul (quando um carro chega, o programa verifica se há vaga). Quando a lotação é máxima, o carro que chega deve ser posicionado em uma fila de espera.

Entrada:

A entrada representa um caso de teste que contém na primeira linha um número inteiro **N** ($0 < N \leq 50$) representando o número de vagas do estacionamento e, em sequência linha por linha, a relação de placas de carro desse caso de teste, além da respectiva demanda para cada placa, naquele instante, separada da placa por um espaço em branco. As possíveis demandas: **E** (entrar no autoestacionamento) ou **S** (sair do autoestacionamento). Haverá no máximo 500 operações de entrada ou saída do estacionamento, e somente acontece operação **S** se houver antes a operação **E** da mesma placa, e nenhuma outra operação **S** pendente. A entrada termina com o final de arquivo.

Saída:

Seu programa deve imprimir um *log* (um histórico) que contenha as ocorrências na ordem de todas as entradas e saídas de carro. Ou seja, na saída do programa, será gerada a sequência dos instantes da fila em linha por linha, com as placas na horizontal representando o sentido Norte–Sul para cada instante. Deve ser expresso “VAZIA” em cada momento em que o estacionamento estiver desocupado.

Exemplo de entrada:

```
3
BMW8594 E
LDU9214 E
BOC4555 E
XYZ4789 E
KKK7777 E
LDU9214 S
KKK7777 S
BOC4555 S
BMW8594 S
XYZ4789 S
```

Saída do exemplo de entrada:

```
BMW8594
BMW8594 LDU9214
BMW8594 LDU9214 BOC4555
BOC4555 BMW8594 XYZ4789
BMW8594 XYZ4789
XYZ4789
VAZIA
```

Problema I
Inveterados na jogatina
Arquivo: sorte.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

Um jogo de dados funciona da seguinte maneira: a cada jogada, o jogador aposta um valor em dinheiro e depois lança dois dados, e o crupiê (funcionário do cassino) a seguir lança dois dados também. Se o jogador acertar o valor exato dos dois dados, ganha 6 vezes o valor apostado; se acerta a soma, ganha 3 vezes o valor apostado. Se a paridade da **SOMA** dos dados do jogador é igual à paridade da **SOMA** dos dados do crupiê, o jogador ganha de volta metade do valor apostado (Exemplo: a soma dos dados do jogador e a soma dos do crupiê são ambas ímpares). Os “deltas” não são acumulativos, ou seja, o jogador só pode ter o saldo na jogada por um dos três critérios possíveis (sendo este o de valor mais favorável ao jogador). Bora implementar o jogo e vencer a ProgBASE 2019? :)

Entrada:

São vários casos de teste. Cada caso de teste contém três linhas: 1) O valor apostado representado por um inteiro não maior que 10^5 ; 2) Os valores dos dois dados lançados pelo jogador, separados por um espaço em branco (as seis faces de um dado típico apresentam cada uma um valor inteiro de 1 a 6, sem repetição); 3) Os valores dos dois dados lançados pelo crupiê, separados por um espaço em branco. A entrada termina com o final de arquivo.

Saída:

Para cada caso de teste, seu programa deve imprimir uma linha com o valor, representado por um número inteiro, resultante da jogada para as finanças do jogador, segundo os critérios (inclusive, em caso de perda na jogada, mesmo parcial, deve ser impresso o valor do saldo **da jogada** com sinal negativo).

Exemplo de entrada:

```
50
2 3
3 2
20
6 4
3 5
30
6 4
3 2
```

Saída do exemplo de entrada:

```
300
-10
-30
```

Problema J
Ji-Paraná tem Paraná
Arquivo: jiparana.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

Há um algoritmo de encriptação inovador que é efetivo graças à inserção de uma cadeia escondida entre os caracteres de outra cadeia. Devido ao fato de que os direitos sobre a novidade tecnológica ainda não estão estabelecidos em portaria, não devemos nos ater em detalhe algum sobre como as cadeias são geradas. Contudo, a fim de validar o algoritmo, é necessário programar um código que verifique se a mensagem original está escondida certo em uma cadeia resultante. Sejam duas cadeias **S** e **T**, tem que ser decidido se **S** é uma subcadeia de **T**, vale por dizer, se é possível “retirar” um subconjunto de caracteres de **T**, de forma que se obtenham todos os de **S** concatenados **na ordem em que estiverem em T**.

Entrada:

São vários casos de teste. Cada caso de teste é caracterizado pelas cadeias **S** e **T**, separadas por um espaço em branco e **contendo caracteres ASCII**. O tamanho máximo de cada cadeia **T** é de 10^5 caracteres. A entrada termina com o final de arquivo.

Saída:

Para cada caso de teste, imprima “Jogou bonito” indicando que a cadeia **S** é subcadeia de **T** segundo o enunciado, ou “Para com isso”, caso contrário.

Exemplo de entrada:

```
sequence subsequence
person compression
VERDI vivaVittorioEmanueleReDiItalia
caseDoesMatter CaseDoesMatter
```

Saída do exemplo de entrada:

```
Jogou bonito
Para com isso
Jogou bonito
Para com isso
```

Problema K
K faltando na conta
Arquivo: k.[c|cpp|java|py2|py3]
Limite de tempo: 1 s

O Problema:

A média aritmética de três inteiros **I**, **J** e **K** é $(I + J + K)/3$. A mediana de três inteiros é, desses, aquele que se encontra entre os outros dois quando todos estão ordenados. Sejam dois inteiros **I** e **J**, qual é **o menor inteiro K** que garante a igualdade entre a média aritmética e a mediana dos três? Hem? **Hem?** Diz aí.

Entrada:

São vários casos de teste. Cada caso de teste é caracterizado por uma linha com os inteiros **I** e **J**, separados por um espaço em branco ($1 \leq I \leq J \leq 10^9$). A entrada termina com uma linha contendo dois zeros, separados por um espaço em branco.

Saída:

Para cada caso de teste, imprima uma linha contendo o menor inteiro **K** que garante a igualdade entre a média aritmética e a mediana de **I**, **J** e **K**.

Exemplo de entrada:

```
1 2
6 10
1 1000000000
0 0
```

Saída do exemplo de entrada:

```
0
2
-999999998
```